

Iterative Control Structures - Questions

Question: Hello For-loop

Short Humorous Scenario: Martin har netop fundet ud af, at hans printer er gået i stykker og i stedet for at udskrive en side, udskriver den bare tal... rigtig mange tal. Han har bedt jer om at lave et program, der kan gøre det samme i terminalen, så han kan føle sig lidt bedre tilpas. *"Kan I ikke bare få den til at skrive alle tal fra 0 til 100? Det er sådan lidt beroligende at se tal rulle forbi..."*

Hvad de allerede ved:

- `main()` funktion
- Variabeldeklaration: `int`
- `for`-loop
- `printf` med format specifiers: `%d`

Opgaven:

1. Åbn VS Code og opret en ny projektmappe
2. Opret en ny fil kaldet `hello_for_loop.c`
3. Definer `main()` funktion
4. I `main()`:
 - a. Deklarer en integer variabel `i` initialiseret til 0
 - b. Lav et `for`-loop som kører mens `i` er mindre end 100
 - c. Indeni loopets kodeblok `{ }` udskriver du værdien af `i` på formatet: `Tal: X` d. Inkrementer `i` med 1 i hver iteration
5. Kør programmet og bekræft outputtet viser: `Tal: 0, Tal: 1, Tal: 2, ... Tal: 99`
6. **PBL:** Martin spørger: *"Hvad sker der, hvis I ændrer startværdien til 1, og betingelsen til `i <= 100`? Hvilke tal bliver udskrevet nu, og hvorfor?"*

Hint:

- Start med at deklare `int i = 0;`
- Brug `i < 100` som betingelse
- Brug `i++` til at inkrementere
- Brug `printf("Tal: %d\n", i);`
- Husk `#include <stdio.h>`

Hvad du aldrig må gøre:

- Glemme at inkrementere iteratoren
- Bruge `i = i + 1` i betingelsen
- Glemme at inkludere `stdio.h`

Question: While Loop

Short Humorous Scenario: Martin har set et gammelt program, der bruger en `for`-loop, og han insisterer på, at *"alt kan gøres på en anden måde!"* Han har udfordret jer til at omskrive det til en `while`-loop. *"Jeg ved god, at det kan det samme, men jeg vil se det med mine egne øjne!"*

Hvad de allerede ved:

- `for`-loop struktur
- `while`-loop struktur
- Variabeldeklaration og scope
- `printf` funktion

Opgaven:

1. Start med følgende program:

```
#include <stdio.h>
int main() {
    for (int a = 10; a <= 20; a++) {
        printf("value of a: %d\n", a);
    }
    return 0;
}
```

2. Omskriv programmet til at bruge en `while`-loop i stedet for `for`-loop
3. Output skal forblive nøjagtig det samme
4. Besvar: Hvad er første værdi den printer ud? Hvad er sidste værdi den printer ud?
5. **PBL:** Martin spørger: *"Kan I også ændre det, så det tæller ned fra 20 til 10 i stedet? Output skal være: value of a: 20, value of a: 19, ... value of a: 10"*

Hint:

- Definer `int a = 10;` udenfor loopet
- Brug `while (a <= 20)` som betingelse
- Flyt inkrementeringen `a++` som sidste statement i loopets kodeblok
- Første værdi: 10, sidste værdi: 20

Hvad du aldrig må gøre:

- Glemme at inkrementere `a` inde i loopet
- Bruge `for` loop strukturen
- Ændre output formatet

Question: itr1

Short Humorous Scenario: Martin har en gammel regnemaskine, der kun kan tælle ned. Han har bedt jer om at lave et program, der kan det samme. *"Jeg har brug for at se tallene 5, 4, 3, 2, ... ned til -5. Bare for at mindes de gode gamle dage, hvor tingene var enklere!"*

Hvad de allerede ved:

- `for`-loop
- Variabeldeklaration: `int`
- `printf` funktion
- Inkrementeringsoperator: `++`
- Dekrementeringsoperator: `--`

Opgaven:

1. Opret en ny fil kaldet `itr1.c`
2. I `main()`:
 - a. Deklarer en integer variabel `i` initialiseret til 5
 - b. Lav et loop som tæller 1 ned i hver iteration
 - c. Loopet skal køre indtil `i` er mindre end -5 d. Udskriv værdien af `i` i hver iteration
3. Kør programmet og bekræft outputtet viser: 5 4 3 2 1 0 -1 -2 -3 -4 -5
4. **PBL:** Martin spørger: *"Hvad sker der, hvis I starter ved 10 i stedet for 5, men holder de andre betingelser? Hvilke tal bliver printet?"*

Hint:

- Brug `int i = 5;` som startværdi
- Brug `i--` til at tælle ned
- Brug `i >= -5` som betingelse (eller `i > -6`)
- Husk at inkludere `stdio.h`

Hvad du aldrig må gøre:

- Bruge `i++` i stedet for `i--`
- Glemme at udskrive værdien
- Bruge forkert betingelse som vil medføre uendeligt loop

Question: itr2

Short Humorous Scenario: Martin har opdaget, at han kun kan lide lige tal. *"Alle ulige tal er for uordentlige!"* han siger. Han har bedt jer om at lave et program, der kun udskriver de tal, han kan lide.

Hvad de allerede ved:

- `for`-loop
- `while`-loop
- Modulo operator: `%`
- Variabeldeklaration: `int`

- `printf` funktion

Opgaven:

1. Opret en ny fil kaldet `itr2.c`
2. I `main()`:
 - a. Lav et loop som itererer gennem alle tal fra 0 til 10
 - b. Udskriv kun de lige tal i dette interval
3. Kør programmet
4. Sammenlign din løsning med dine medstuderende
5. **PBL:** Martin spørger: "Kan I ændre programmet til at udskrive alle ulige tal i stedet? Og hvordan ville I gøre det, uden at bruge modulo?"

Hint:

- Lige tal er divisibile med 2: `i % 2 == 0`
- Du kan starte ved 0 og inkrementere med 2: `i += 2`
- Begge løsninger er korrekte

Hvad du aldrig må gøre:

- Udskrive ulige tal
 - Glemme at inkludere `stdio.h`
-

Question: esp32_1

Short Humorous Scenario: Martin har fået en ny ESP32 og er fascineret af, at den kan tælle. "Den gamle regnemaskine kunne kun tælle ned, men denne her kan gøre det i begge retninger! Lav et program, der tæller op fra 0 til 10 i setup, men kun de lige tal!"

Hvad de allerede ved:

- `void setup()` funktion
- `for`-loop
- `Serial.begin()`
- `Serial.print()` / `Serial.println()`
- ESP32 IDE (Arduino IDE)

Opgaven:

1. I Arduino IDE:
 - a. Lav et nyt projekt kaldet `esp32_1`
2. I `setup()`:
 - a. Start seriell kommunikation med `Serial.begin(115200)`

- b. Lav et loop, som tæller tallene 0, 2, 4, 6, 8, 10
- c. Udskriv hver værdi til Serial Monitor

3. Upload og kørs programmet

4. **PBL:** Martin spørger: *"Kan I ændre programmet til at tælle op fra 0 til 20, men kun udskrive hver tredje værdi (0, 3, 6, 9, ...)?"*

Hint:

- Start loopet ved `i = 0`
- Brug inkrement `i += 2` for kun at få lige tal
- Brug `Serial.println()` til at udskrive med ny linje
- Husk at inkludere `delay(100)` for at kunne se outputtet

Hvad du aldrig må gøre:

- Glemme `Serial.begin()`
- Udskrive ulige tal
- Glemme at upload programmet til ESP32

Question: esp32_2

Short Humorous Scenario: Martin har bemærket, at hans forrige program kun kørte én gang. *"Det er som om det gik i stå efter setup! Kan I ikke få det til at fortsætte med at tælle?"* han spørger forvirret.

Hvad de allerede ved:

- `void setup()` funktion
- `void loop()` funktion
- `for`-loop
- `Serial.begin()`
- `Serial.print()`

Opgaven:

1. Tag udgangspunkt i forrige opgave (`esp32_1`)
2. Flyt `for`-loopet fra `setup()` til `void loop()`
3. Upload og kørs programmet
4. Observer outputtet i Serial Monitor
5. **PBL:** Martin spørger: *"Hvad sker der, hvis I flytter `Serial.begin()` ind i `loop()`? Hvorfor er det en dårlig idé?"*

Hint:

- `void loop()` kører kontinuerligt på ESP32
- Hold `Serial.begin()` i `setup()`
- Du behøver ikke at kalde `Serial.begin()` mere end én gang

Hvad du aldrig må gøre:

- Kalde `Serial.begin()` inde i `loop()`
 - Glemme at flytte loopet til `void loop()`
 - Glemme at inkludere `both setup()` og `loop()`
-

Question: loops2

Short Humorous Scenario: Martin har fundet et gammelt program, der udskriver primtal, men det ser ud som en eneste lang linje kode. *"Jeg kan slet ikke læse det her! Kan I ikke gøre det mere overskueligt?"*

Hvad de allerede ved:

- `for`-loop
- `if`-betingelser
- Nested kontrolstrukturer
- Debugger (F5 og F10)
- Breakpoints

Opgaven:

Programmet herunder udskriver alle primtal op til 100. Omskriv det med Tuborg-klammer `{}` og noter/comments.

```
#include <stdio.h>

int main () {

    /* local variable definition */
    int i, j;

    for(i = 2; i<100; i++) {

        for(j = 2; j <= (i/j); j++)
            if(!(i%j)) break; // if factor found, not prime
        if(j > (i/j)) printf("%d is prime\n", i);
    }

    return 0;
}
```

1. Opgave a: Forstå hvordan programmet virker

- a. Analyser flowet af koden

- b. Identificer hvilke loops og conditional-structurer er nested under hinanden
- c. Tilføj de manglende Tuborg-klammer `{ }` d. Tilføj kommentarer, der forklarer hver sektion

2. **Opgave b (DEBUGGING):** Hvor mange gange har vi været inde i j-loopet's codeblok, når $i = 23$?

- a. Benyt en `int counter = 0;`
- b. Inkrementer `counter++` hver gang man kommer ind i j-loop codeblokket
- c. Udskriv counter når $i = 23$

3. **PBL:** Martin spørger: "Kan I modificere programmet til at udskrive alle primtal op til 200 i stedet for 100?"

Hint:

- Brug debuggeren med F5 og F10 til at træde gennem koden
- Sæt breakpoints for at se flowet
- Start med at tilføje klammer til det yderste loop først
- counter skal placeres inde i det inderste loop

Hvad du aldrig må gøre:

- Ændre den originale logik
- Fjerne nødvendige betingelser
- Glemme at erklære counter variablen

Question: Loops3

Short Humorous Scenario: Martin har lavet en chatbot til sin ESP32, men den forstår kun én ting ad gangen. "Den er lidt dum, den her. Kan I lave et program, der kan forstå mere end én kommando?" han spørger. "Når jeg skriver 'y' skal den sige 'hej med dig', og når jeg skriver 'q' skal den sige 'farvel med dig' og lukke!"

Hvad de allerede ved:

- `char` variabeltype
- `printf` funktion
- `scanf` funktion
- `switch`-statement
- `while`-loop

Opgaven:

1. Opret en ny fil kaldet `communicate.c`
2. Definer en `char c` variabel
3. Lav programmet til at udskrive "waiting for input: "
4. Modtag ét char input fra brugeren ved hjælp af `scanf(" %c", &c);`
5. Lav et `switch`-statement, som:

- a. Skriver "hej med dig" hvis `c` er 'y' eller 'Y'
 - b. Skriver "Farvel med dig" og lukker programmet hvis `c` er 'q' eller 'Q'
 - c. Skriver "Hvad vil du egentlig?" for alle andre input
6. Indsæt hele `printf`, `scanf`, og `switch`-statementet i et `while`-loop
7. `while`-loopet skal stoppe, når brugeren indtaster 'q' eller 'Q'
8. Kør og test programmet
9. **PBL:** Martin spørger: *"Kan I udvide programmet til også at håndtere kommandoen 'h' for help, der udskriver en hjælpetekst med de tilgængelige kommandoer?"*

Hint:

- Læs dokumentationen for `scanf`
- Læs dokumentationen for `switch` statements
- Husk at inkludere `break;` i hver case
- Brug `return 0;` til at afslutte programmet
- Mellemrummet i `" %c"` i `scanf` springer whitespace over

Hvad du aldrig må gøre:

- Glemme `break` statements i `switch`
- Glemme at inkludere `stdio.h`
- Glemme `&` foran variablen i `scanf`